Kirin

Chess Engine Documentation

Strydr Silverberg
strydr_silverberg@mines.edu

Colorado School of Mines

April 24, 2026

Kirin: A high-performance chess engine written in C++ powering an autonomous chess system

2026 Colorado School of Mines ProtoFund Recipient

Team: Strydr Silverberg, Colin Dake

Contents

1 Introduction **4**
1.1 Overview . . . 4
1.2 Features . . . 4
1.3 System Requirements . . . 4

2 Installation **5**
2.1 Building from Source . . . 5
2.2 Pre-built Binaries . . . 5
2.3 Start Playing . . . 5

3 Architecture **6**
3.1 Overview . . . 6
3.2 Board Representation . . . 6
3.2.1 Bitboard Data Structures . . . 6
3.2.2 Square Encoding . . . 6
3.2.3 Piece Encoding . . . 7
3.2.4 Castling Rights . . . 7
3.2.5 Bit Manipulation Macros . . . 7
3.2.6 Chess Position Example . . . 7
3.3 Move Generation . . . 8
3.3.1 Move Encoding . . . 8
3.3.2 Attack Tables . . . 8
3.3.3 Magic Bitboards . . . 8
3.3.4 Legal Move Generation . . . 9

4 Search Algorithm **10**
4.1 Overview . . . 10
4.2 Alpha-Beta Search . . . 10
4.3 Iterative Deepening . . . 10
4.4 Search Enhancements . . . 11
4.4.1 Transposition Table . . . 11
4.4.2 Move Ordering . . . 11
4.4.3 Repetition Detection . . . 12

5 Evaluation **13**
5.1 Overview . . . 13
5.2 Material . . . 13
5.3 Positional Factors . . . 13
5.3.1 Piece-Square Tables . . . 13
5.3.2 Knight Positioning . . . 13
5.4 Pawn Structure . . . 14
5.4.1 Double Pawns . . . 14

5.4.2 Isolated Pawns . . . 14
5.4.3 Passed Pawns . . . 14

6 UCI Protocol **15**
6.1 Overview . . . 15
6.2 Supported Commands . . . 15
6.2.1 Standard UCI Commands . . . 15
6.2.2 Engine Responses . . . 15
6.3 Skill Level . . . 16
6.3.1 Difficulty Tiers . . . 16
6.3.2 UCI Configuration . . . 16
6.3.3 Implementation . . . 16

7 API Reference **18**
7.1 Core Functions . . . 18
7.1.1 Board Management . . . 18
7.1.2 Move Generation . . . 18
7.1.3 Search . . . 18
7.1.4 Evaluation . . . 19

8 Testing **20**
8.1 Perft Testing . . . 20
8.2 Test Positions . . . 20

9 Performance **21**
9.1 Optimization Techniques . . . 21
9.2 Memory Usage . . . 21

10 Contributing **22**
10.1 Development Setup . . . 22
10.2 Code Style . . . 22
10.3 Pull Request Process . . . 22

A Glossary **23**

B References **24**

Chapter 1

Introduction

1.1 Overview

This document provides comprehensive documentation for the **Kirin Chess Engine**. Kirin is a chess engine written in C++ and an integral part of the **Kirin** autonomous chess system.

Kirin is the computational engine powering an autonomous chess system designed to deliver a complete single-player experience. By integrating intelligent move generation with a physical board capable of independent piece manipulation, the system eliminates the need for a human opponent while preserving the tactile satisfaction of over-the-board play.

1.2 Features

- **Bitboard Representation**: Efficient 64-bit integer board representation for fast move generation and evaluation
- **Magic Bitboards**: Pre-computed attack tables for sliding pieces (bishops and rooks) using magic number multiplication
- **Zobrist Hashing**: Position hashing for transposition table lookups and repetition detection
- **Alpha-Beta Search**: Negamax search with alpha-beta pruning and iterative deepening
- **UCI Protocol**: Full Universal Chess Interface compatibility for GUI integration
- **Advanced Evaluation**: Material counting, piece-square tables, pawn structure analysis
- **Move Ordering**: MVV-LVA, killer moves, and history heuristic for search efficiency
- **Time Management**: Configurable time controls with lag compensation

1.3 System Requirements

Component	Requirement
Operating System	Linux (primary development target)
Memory	Minimum 256 MB RAM
Processor	Any modern x86-64 CPU
Compiler	GCC or Clang with C++17 support
Dependencies	CMake 3.10+, POSIX headers (`unistd.h`, `sys/time.h`)

Table 1.1: Minimum system requirements

Chapter 2

Installation

2.1 Building from Source

Clone the repository and build:

```
git clone git@github.com:strvdr/kirin-autonomous-chess.git
cd kirin-autonomous-chess/kirin
cmake -S . -B build
cmake --build build
./build/kirin
```

2.2 Pre-built Binaries

Build from source is the primary supported workflow for the current repository state.

2.3 Start Playing

Kirin is a UCI compatible chess engine, so configuring something like Lichess Bot is very simple. I have also created a chess CLI tool for viewing and playing games called zduel which you can use to play games against Kirin from the terminal.

If you want to play around with the engine but not an actual game, run the compiled binary and try this:

```
kirin >> id name Kirin
kirin >> author name Strydr Silverberg
kirin >> uciok

user >> ucinewgame
user >> isready

kirin >> readyok

user >> go depth 12
```

Chapter 3

Architecture

3.1 Overview

Kirin uses a bitboard-based architecture optimized for modern 64-bit processors. The engine is structured around several core components: board representation, move generation, position evaluation, and tree search.

3.2 Board Representation

3.2.1 Bitboard Data Structures

The board is represented using 12 bitboards (one per piece type and color) plus 3 occupancy bitboards:

```
// Define Bitboard Type
#define U64 unsigned long long

// Piece bitboards: P, N, B, R, Q, K, p, n, b, r, q, k
U64 bitboards[12];

// Occupancy bitboards: white, black, both
U64 occupancy[3];

// Game state
int side;               // Side to move (white=0, black=1)
int enpassant;          // En passant target square
int castle;             // Castling rights (4 bits)
U64 hashKey;            // Zobrist hash of position
```

3.2.2 Square Encoding

Squares are encoded as integers 0-63, with a8=0 and h1=63:

```
enum {
  a8, b8, c8, d8, e8, f8, g8, h8,
  a7, b7, c7, d7, e7, f7, g7, h7,
  a6, b6, c6, d6, e6, f6, g6, h6,
  a5, b5, c5, d5, e5, f5, g5, h5,
  a4, b4, c4, d4, e4, f4, g4, h4,
  a3, b3, c3, d3, e3, f3, g3, h3,
  a2, b2, c2, d2, e2, f2, g2, h2,
  a1, b1, c1, d1, e1, f1, g1, h1, no_sq
};
```

3.2.3 Piece Encoding

Pieces are encoded as integers 0-11:

```
enum { P, N, B, R, Q, K, p, n, b, r, q, k };
```

3.2.4 Castling Rights

Castling rights are stored as a 4-bit value:

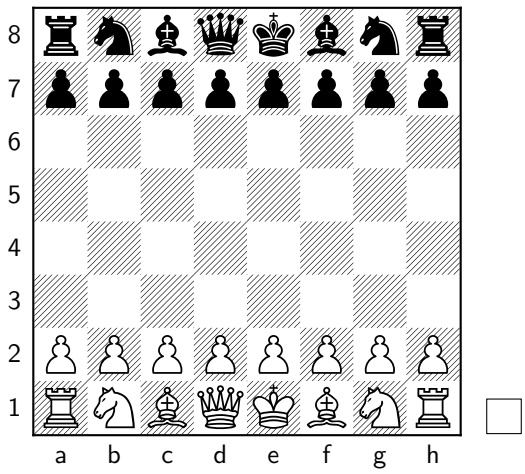
```
// Binary encoding:
// 0001 (1) = white king-side
// 0010 (2) = white queen-side
// 0100 (4) = black king-side
// 1000 (8) = black queen-side
enum { wk = 1, wq = 2, bk = 4, bq = 8 };
```

3.2.5 Bit Manipulation Macros

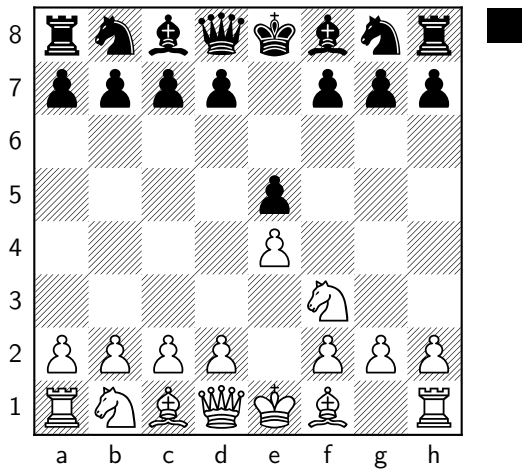
```
#define setBit(bitboard, square) ((bitboard) |= (1ULL << (square)))
#define getBit(bitboard, square) ((bitboard) & (1ULL << (square)))
#define popBit(bitboard, square) ((bitboard) &= ~(1ULL << (square)))
```

3.2.6 Chess Position Example

Here is an example starting position:



After `1.e4 e5 2.Nf3`:



3.3 Move Generation

3.3.1 Move Encoding

Moves are encoded as 24-bit integers containing all move information:

```
// Binary move representation:
// 0000 0000 0000 0000 0011 1111  source square      0x3f
// 0000 0000 0000 1111 1100 0000  target square      0xfc0
// 0000 0000 1111 0000 0000 0000  piece              0xf000
// 0000 1111 0000 0000 0000 0000  promoted piece     0xf0000
// 0001 0000 0000 0000 0000 0000  capture flag       0x100000
// 0010 0000 0000 0000 0000 0000  double pawn push   0x200000
// 0100 0000 0000 0000 0000 0000  en passant flag    0x400000
// 1000 0000 0000 0000 0000 0000  castling flag      0x800000

#define encodeMove(source, target, piece, promoted, capture, dpp,
    enpassant, castle) \
  (source) | (target << 6) | (piece << 12) | (promoted << 16) | \
  (capture << 20) | (dpp << 21) | (enpassant << 22) | (castle << 23)
```

3.3.2 Attack Tables

Leaper pieces (pawns, knights, kings) use pre-computed attack tables:

```
U64 pawn_attacks[2][64];    // [side][square]
U64 knightAttacks[64];
U64 kingAttacks[64];
```

3.3.3 Magic Bitboards

Sliding pieces (bishops, rooks, queens) use magic bitboard technique for O(1) attack generation:

```
U64 bishopMasks[64];
U64 rookMasks[64];
U64 bishopAttacks[64][512];    // 32KB
U64 rookAttacks[64][4096];     // 256KB
U64 bishopMagicNums[64];
U64 rookMagicNums[64];
```

```

static inline U64 getBishopAttacks(int square, U64 occupancy) {
  occupancy &= bishopMasks[square];
  occupancy *= bishopMagicNums[square];
  occupancy >>= 64 - bishopRelevantBits[square];
  return bishopAttacks[square][occupancy];
}
```

3.3.4 Legal Move Generation

Algorithm 1 Legal Move Generation

```
procedure GenerateMoves(position)
    moves ← empty list
    for each piece type do
        bitboard ← pieces of this type
        while bitboard has pieces do
            square ← get least significant bit index
            Generate pseudo-legal moves for piece at square
            for each pseudo-legal move do
                Make move on board
                if own king not in check then
                    Add move to list
                end if
                Restore board state
            end for
            Remove bit from bitboard
        end while
    end for
    return moves
end procedure
```

Chapter 4

Search Algorithm

4.1 Overview

Kirin uses a negamax search with alpha-beta pruning, iterative deepening, and aspiration windows.

4.2 Alpha-Beta Search

Algorithm 2 Alpha-Beta with Negamax

```
function AlphaBeta(position, depth, α, β)
    if depth = 0 then
        return QuiescenceSearch(position, α, β)
    end if
    if position is repetition or fifty-move draw then
        return 0
    end if
    Check transposition table for cached result
    for each move in GenerateMoves(position) do
        MakeMove(position, move)
        score ← −AlphaBeta(position, depth−1, −β, −α)
        UnmakeMove(position, move)
        if score ≥ β then
            Store killer move
            return β                                    ▷ Beta cutoff
        end if
        if score > α then
            α ← score
            Update history heuristic
            Update PV table
        end if
    end for
    Store position in transposition table
    return α
end function
```

4.3 Iterative Deepening

The engine uses iterative deepening with aspiration windows:

```
void searchPosition(int depth) {
  int score = 0;
```

```
  int alpha = -infinity;
  int beta = infinity;

  for(int currentDepth = 1; currentDepth <= depth; currentDepth++) {
    score = negamax(alpha, beta, currentDepth);

    // Aspiration window for next iteration
    alpha = score - 50;
    beta = score + 50;

    // Print UCI info
    printf("info score cp %d depth %d nodes %ld time %d pv ...",
           score, currentDepth, nodes, getTimeMS() - startTime);
  }
  printf("bestmove ...");
}
```

4.4 Search Enhancements

4.4.1 Transposition Table

Zobrist hashing is used for position identification:

```
U64 pieceKeys[12][64];     // Random keys for each piece on each square
U64 enpassantKeys[64];     // Random keys for en passant squares
U64 castlingKeys[16];      // Random keys for castling states
U64 sideKey;               // Random key for side to move

U64 generateHashKey() {
  U64 finalKey = 0ULL;
  for(int piece = P; piece <= k; piece++) {
    U64 bitboard = bitboards[piece];
    while(bitboard) {
      int square = getLSBindex(bitboard);
      finalKey ^= pieceKeys[piece][square];
      popBit(bitboard, square);
    }
  }
  if(enpassant != no_sq) finalKey ^= enpassantKeys[enpassant];
  finalKey ^= castlingKeys[castle];
  if(side == black) finalKey ^= sideKey;
  return finalKey;
}
```

4.4.2 Move Ordering

Moves are ordered for optimal alpha-beta pruning:

1. Hash move (from transposition table)
2. Winning captures (MVV-LVA ordering)
3. Killer moves (quiet moves that caused beta cutoffs)
4. History heuristic (successful quiet moves)
5. Remaining quiet moves

MVV-LVA Table

Most Valuable Victim - Least Valuable Attacker scoring:

```
// MVV LVA [attacker][victim]
static int mvvLVA[12][12] = {
  105, 205, 305, 405, 505, 605, ...  // Pawn captures
  104, 204, 304, 404, 504, 604, ...  // Knight captures
  103, 203, 303, 403, 503, 603, ...  // Bishop captures
  102, 202, 302, 402, 502, 602, ...  // Rook captures
  101, 201, 301, 401, 501, 601, ...  // Queen captures
  100, 200, 300, 400, 500, 600, ...  // King captures
};
```

4.4.3 Repetition Detection

The engine maintains a repetition table to detect threefold repetition:

```
U64 repetitionTable[1000];
int repetitionIndex;

static inline int isRepetition() {
  for(int i = 0; i < repetitionIndex; i++) {
    if(repetitionTable[i] == hashKey) return 1;
  }
  return 0;
}
```

Chapter 5

Evaluation

5.1 Overview

The evaluation function returns a score in centipawns from the perspective of the side to move. Positive scores favor the side to move.

5.2 Material

Piece	Value (centipawns)
Pawn	100
Knight	300
Bishop	350
Rook	500
Queen	1000
King	10000

Table 5.1: Piece values in Kirin

5.3 Positional Factors

5.3.1 Piece-Square Tables

Each piece type has a positional bonus table. Example for pawns:

```
const int pawnScore[64] = {
    90,  90,  90,  90,  90,  90,  90,  90,
    30,  30,  30,  40,  40,  30,  30,  30,
    20,  20,  20,  30,  30,  30,  20,  20,
    10,  10,  10,  20,  20,  10,  10,  10,
     5,   5,  10,  20,  20,   5,   5,   5,
     0,   0,   0,   5,   5,   0,   0,   0,
     0,   0,   0, -10, -10,   0,   0,   0,
     0,   0,   0,   0,   0,   0,   0   0
};
```

5.3.2 Knight Positioning

Knights are rewarded for central placement:

```
const int knightScore[64] = {
    -5,   0,   0,   0,   0,   0,   0,  -5,
    -5,   0,   0,  10,  10,   0,   0,  -5,
    -5,   5,  20,  20,  20,  20,   5,  -5,
```

```
    -5,  10,  20,  30,  30,  20,  10,  -5,
    -5,  10,  20,  30,  30,  20,  10,  -5,
    -5,   5,  20,  10,  10,  20,   5,  -5,
    -5,   0,   0,   0,   0,   0,   0,  -5,
    -5, -10,   0,   0,   0,   0, -10,  -5
};
```

5.4 Pawn Structure

5.4.1 Double Pawns

Doubled pawns receive a penalty:

```
const int doublePawnPenalty = -10;

doublePawns = countBits(bitboards[P] & fileMasks[square]);
if(doublePawns > 1) score += doublePawns * doublePawnPenalty;
```

5.4.2 Isolated Pawns

Pawns with no friendly pawns on adjacent files are penalized:

```
const int isolatedPawnPenalty = -10;

if((bitboards[P] & isolatedMasks[square]) == 0)
    score += isolatedPawnPenalty;
```

5.4.3 Passed Pawns

Passed pawns receive bonuses based on how advanced they are:

```
const int passedPawnBonus[8] = { 0, 10, 30, 50, 75, 100, 150, 200 };

if((passedWhiteMasks[square] & bitboards[p]) == 0)
    score += passedPawnBonus[getRank[square]];
```

Chapter 6

UCI Protocol

6.1 Overview

The engine communicates via the Universal Chess Interface (UCI) protocol.

6.2 Supported Commands

6.2.1 Standard UCI Commands

IN `uci`
Initialize UCI mode and request engine identification.

IN `isready`
Check if engine is ready. Engine responds with `readyok`.

IN `ucinewgame`
Reset engine state for a new game.

IN `position [fen | startpos] moves ...`
Set up a position.

IN `go [options]`
Start calculating. Options include:

- `depth n` – search to depth n
- `movetime ms` – search for ms milliseconds
- `wtime/btime ms` – time remaining for white/black
- `winc/binc ms` – increment per move
- `movestogo n` – moves until next time control
- `infinite` – search until stopped
- `perft n` – run perft test to depth n

IN `stop`
Stop calculating.

IN `quit`
Exit the engine.

6.2.2 Engine Responses

```
id name Kirin
id author Strydr Silverberg
uciok

info score cp 35 depth 8 nodes 234567 time 1250 pv e2e4 e7e5 g1f3

bestmove e2e4
```

6.3 Skill Level

Kirin exposes a three-tier difficulty setting that adjusts both search depth and move quality, making the engine accessible to players of all levels while preserving full strength for competitive play.

6.3.1 Difficulty Tiers

Level	Name	Max Search Depth	Move Noise
0	Easy	3 plies	±150 centipawns
1	Medium	5 plies	±50 centipawns
2	Hard (default)	Caller-supplied depth	None

Table 6.1: Skill level configuration

Two independent mechanisms are combined to simulate human-like play at lower difficulty settings:

1. **Depth capping.** Inside `searchPosition`, the caller-supplied depth is silently clamped to a tier-specific ceiling before iterative deepening begins. At skill 0 the engine never searches deeper than 3 plies; at skill 1 the cap is 5 plies; at skill 2 the cap is set to 64 (effectively unlimited for practical time controls).

2. **Root-move noise.** After the search completes, the engine's best move is not always played. At skill levels below 2, each legal root move is assigned the engine's score perturbed by a random offset drawn uniformly from $[-\mathit{range}, +\mathit{range}]$, where *range* is 150 cp at skill 0 and 50 cp at skill 1. The move with the highest *noisy* score is selected. This causes the engine to occasionally prefer a suboptimal move in a way that resembles human error rather than purely random play.

6.3.2 UCI Configuration

The skill level is exposed as a standard UCI spin option and can be set from any compatible GUI or from the command line:

```
setoption name Skill Level value 0   % Easy
setoption name Skill Level value 1   % Medium
setoption name Skill Level value 2   % Hard (default)
```

The engine announces the option during UCI initialisation:

```
option name Skill Level type spin default 2 min 0 max 2
```

Values outside the range $[0, 2]$ are clamped silently. The engine acknowledges each change with an info string:

```
info string Skill Level set to 1
```

6.3.3 Implementation

The global variable `skillLevel` in `search.cpp` stores the current setting. It is read in two places:

```
// Depth cap applied at the start of searchPosition()
static const int skillDepthCap[3] = { 3, 5, 64 };
int depthCap = skillDepthCap[skillLevel];
if (depth > depthCap) depth = depthCap;

// Noise injection applied after the best move is found
if (skillLevel < 2) {
    static const int noiseRange[2] = { 150, 50 };
    int range = noiseRange[skillLevel];
    // ... re-score root moves with random perturbation
}
```

Note: Skill level does not affect the correctness of move generation or position evaluation — illegal moves are never produced regardless of the difficulty setting. The engine test suite (`engine_test.cpp`) verifies this invariant across all three levels and multiple board positions.

Chapter 7

API Reference

7.1 Core Functions

7.1.1 Board Management

`parseFEN(const char *fen)` *Function*
Parameters: fen – FEN string describing position
Returns: void

Parses a FEN string and sets up the board position, including piece placement, side to move, castling rights, en passant square, and initializes the hash key.

`printBoard()` *Function*
Parameters: None
Returns: void

Prints the current board position to stdout with piece symbols, side to move, castling rights, en passant square, and hash key.

7.1.2 Move Generation

`generateMoves(moves *moveList)` *Function*
Parameters: moveList – pointer to move list structure
Returns: void

Generates all pseudo-legal moves for the current position and stores them in the provided move list.

`makeMove(int move, int moveFlag)` *Function*
Parameters: move – encoded move, moveFlag – allMoves or onlyCaptures
Returns: int – 1 if legal, 0 if illegal

Attempts to make a move on the board. Returns 1 if the move is legal (doesn't leave own king in check), 0 otherwise.

7.1.3 Search

`searchPosition(int depth)` *Function*
Parameters: depth – maximum search depth
Returns: void

Searches the current position using iterative deepening and prints the best move via UCI protocol.

`negamax(int alpha, int beta, int depth)` *Function*
Parameters: alpha, beta – search window, depth – remaining depth
Returns: int – position score

Core search function implementing negamax with alpha-beta pruning.

7.1.4 Evaluation

`evaluate()` *Function*

Parameters: None

Returns: int – position score in centipawns

Returns a static evaluation of the current position from the perspective of the side to move.

`isAttacked(int square, int side)` *Function*

Parameters: square – target square, side – attacking side

Returns: int – 1 if attacked, 0 otherwise

Checks if a square is attacked by the specified side.

Chapter 8

Testing

8.1 Perft Testing

Perft (performance test) validates move generation by counting all leaf nodes at a given depth:

```
position startpos
go perft 6
```

Depth	Nodes
1	20
2	400
3	8,902
4	197,281
5	4,865,609
6	119,060,324

Table 8.1: Perft results from starting position

8.2 Test Positions

The engine includes several built-in test positions:

```
#define startPosition "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w
    KQkq - 0 1"
#define tricky_position "r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/
    PPPBBPPP/R3K2R w KQkq - 0 1"
#define killer_position "rnbqkb1r/pp1p1pPp/8/2p1pP2/1P1P4/3P3P/P1P1P3/
    RNBQKBNR w KQkq e6 0 1"
#define cmk_position "r2q1rk1/ppp2ppp/2n1bn2/2b1p3/3pP3/3P1NPP/PPP1NPB1
    /R1BQ1RK1 b - - 0 9"
```

Chapter 9

Performance

9.1 Optimization Techniques

- **Magic bitboards**: O(1) sliding piece attack generation using pre-computed magic numbers
- **Incremental hash updates**: Zobrist hash is updated incrementally during make/unmake move
- **Copy-make**: Board state is copied before making moves for efficient unmake
- **Inline functions**: Critical functions are marked `static inline` for compiler optimization
- **Bit manipulation intrinsics**: Uses efficient bit counting and LSB detection

9.2 Memory Usage

Component	Size
Bishop attack table	32 KB
Rook attack table	256 KB
Transposition table	Configurable
Repetition table	8 KB

Table 9.1: Memory usage of major components

Chapter 10

Contributing

10.1 Development Setup

1. Fork the repository
2. Clone your fork locally
3. Build with `make debug` for development builds
4. Run tests with `make test`

10.2 Code Style

- Use camelCase for function and variable names
- Use SCREAMING_CASE for macros and constants
- Keep functions focused and under 50 lines where possible
- Comment non-obvious algorithms and magic numbers

10.3 Pull Request Process

1. Fork the repository
2. Create a feature branch
3. Make changes and verify perft results match expected values
4. Submit a pull request with clear description

Appendix A

Glossary

Alpha-Beta A search algorithm that prunes branches that cannot affect the final result.

Bitboard A 64-bit integer representing piece positions on the board, where each bit corresponds to a square.

Centipawn One hundredth of a pawn; standard unit for evaluation scores.

FEN Forsyth-Edwards Notation; a standard for describing chess positions in a single line of text.

Magic Bitboard A technique using multiplication and lookup tables for O(1) sliding piece attack generation.

MVV-LVA Most Valuable Victim - Least Valuable Attacker; a move ordering heuristic for captures.

Negamax A variant of minimax that exploits the zero-sum property of chess.

Perft Performance test; counts all leaf nodes at a given depth to validate move generation.

Ply A half-move; one move by either white or black.

Quiescence Search Extended search of capture sequences to avoid horizon effects.

UCI Universal Chess Interface; standard protocol for chess engine communication.

Zobrist Hashing A technique for hashing chess positions using XOR of random numbers.

Skill Level A configurable difficulty setting (0–2) that limits search depth and injects random noise into move selection to simulate human-like play.

Move Noise A random centipawn perturbation applied to root-move scores at lower skill levels, causing the engine to occasionally prefer a suboptimal move.

Appendix B

References

- Chess Programming Wiki: https://www.chessprogramming.org/
- UCI Protocol Specification: https://gist.github.com/DOBRO/2592c6dad754ba67e6dcaec8c90165bf
- Stockfish source code: https://github.com/official-stockfish/Stockfish
- VICE Chess Engine by Richard Allbert (time control implementation reference)
- Tord Romstad's magic number generation proposal
- Maksim Korzh's chess programming guidance